Q3

1. Main feature: Self-attention lets each token interact with all others using content-dependent weights, giving a global receptive field from the first layer. Convolutions use fixed local filters, so interactions are only nearby and content-independant.

   Main challenge: Self attention needs the full $N \times N$ attention matrix for N tokens:
   Time complexity: $O(N^2 \cdot d)$
   Memory complexity: $O(N^2)$ to store attention scores.
   This quadratic scaling is the obstacle for large images.

2. Given input $X \in \mathbb{R}^{B \times N \times d}$:

   $Q = XW_Q$, $K = XW_K$, $V = XW_V$ ($W \in \mathbb{R}^{d \times d_k}$)

   $S = \frac{QK^T}{\sqrt{d_k}} \in \mathbb{R}^{B \times N \times N}$

   $A = \text{SoftMax}(S)$

   $Y = AV \in \mathbb{R}^{B \times N \times d_k}$

   $O = YW_O \in \mathbb{R}^{B \times N \times d}$ ($W_O \in \mathbb{R}^{d_k \times d}$)

   O is the final linearly projected output

Q4

1. Given $X \in \mathbb{R}^{B \times N \times d}$, number of heads H, head size $d_h = \frac{d}{H}$. For each head h:

   $Q_h = XW_Q^{(h)}$, $K_h = XW_K^{(h)}$, $V_h = XW_V^{(h)}$, ($W^{(h)} \in \mathbb{R}^{d \times d_h}$)

   $A_h = \text{SoftMax}\left(\frac{Q_h K_h^T}{\sqrt{d_h}}\right) \in \mathbb{R}^{B \times N \times N}$

   $Y_h = A_h V_h \in \mathbb{R}^{B \times N \times d_h}$

   Concatenate heads and project:

   $Y = \text{Concat}(Y_1, \cdots, Y_H) \in \mathbb{R}^{B \times N \times d}$, $O = YW_O$, $W_O \in \mathbb{R}^{d \times d}$

Scanné avec CamScanner

Q5

1. Input $X \in \mathbb{R}^{B \times N \times d}$

Self-attention sublayer:

$\tilde{X}_1 = LN_1(X)$, $A = MSA(\tilde{X}_1)$, $X_1 = X + \text{Dropout}(A)$

MLP sublayer:

$\tilde{X}_2 = LN_2(X_1)$, $M = MLP(\tilde{X}_2)$, $Y = X_1 + \text{Dropout}(M)$

Output Y has the same shape as X.

Q6

1. Class token: A learnable vector prepended to the patch sequence. It attends to all patches and becomes a global summary used by the classifier.

Compared with average pooling, [CLS] can learn task-specific aggregation through all layers.

2. Positional embedding: Vector added to encode their positions, without PE, self-attention is permutation-invariant and can't know where patches come from.

Experimental analysis

1. embed_dim: larger → higher capacity and accuracy. but slower and may overfit.

patch_size: smaller → more tokens, better details but slower

larger → faster but less precise.

nb_blocks: deeper → stronger representation, but more compute and possible overfitting.

2. Final performance: on MNIST, ViT can reach > 95%. To improve: use AdamW + weight decay, dropout, and data augmentation.

3. Complexity: self-attention = $O(N^2 \cdot d)$ time and $O(N^2)$ accuracy

To reduce: use lager patches, windowed or sparse attention, or token pruning.

Scanné avec CamScanner

Q8

a). Problem: ViT-S is built for 3-channel 224 × 224 images with absolute positional embeddings sized for a fixed token grid. Feeding MNIST (1×28×28) causes channel mismatch (1 ≠ 3) and resolution / token-grid mismatch

Why: ViT uses a fixed patch embed + absolute PE tied to image size / patch grid. Changing the input size changes the number of tokens, breaking PE and degrading performance.

CNNs: Many CNNs are convolutional + global pooling, so they often handle variable resolutions. ViTs with absolute PE are less resolution-agnostic.

d) The custom small ViT achieved 96.9% test accuracy

The timm ViT-S model adapted to 28×28 MNIST images with ImageNet pretraining obtained only 90.7 – 90.9% accuracy.

Analysis: The pretrained ViT-S did not improve performance

1. Too few tokens: At 28×28 with patch = 16, only 1×1 patch remains, so almost no spatial information is learned.
2. Finetuning setup: Using Adam with default LR (1e-3) is too large for finetuning and may destroy pretrained weights.

Improvements: To benefit from ImageNet pretraining, we can resize MNIST to 224 × 224 and apply ImageNet normalization before training.
We can also use a smaller patch size to increase the number of tokens
In addition, we can finetune with AdamW, smaller learning rate and weight decay.

Scanné avec CamScanner